Complete Guide to Core Distributed Systems Design

High-Level Design (HLD) Reference

01 URL Shortener

02 Rate Limiter

03 Distributed Cache

04 Message Queue (Kafka)

05 Job Scheduler

06 Notification System

07 Search Engine

08 File Storage (S3)

Partitioning · Replication · Consistency · Availability · Fault Tolerance

Complete Guide to Core Distributed Systems Design

High-Level Design (HLD) Reference

01 URL Shortener

02 Rate Limiter

03 Distributed Cache

04 Message Queue (Kafka)

05 Job Scheduler

06 Notification System

07 Search Engine

08 File Storage (S3)

Partitioning · Replication · Consistency · Availability · Fault Tolerance

Complete Guide to Core Distributed Systems Design (HLD)

1. URL Shortener (like Bitly)

Problem Statement

Convert long URLs into short, unique aliases and redirect users when they visit the short URL.

Functional Requirements

```
- Given a long URL → generate a short URL
- When user visits short URL → redirect to original URL
- Custom aliases (optional)
- Link expiration (optional)
- Analytics (click count, location, etc.)
```

Non-Functional Requirements

```
- Very low latency for redirection (<100ms)
- High availability (redirects should never fail)
- Short URLs should not be predictable
- Read-heavy system (100:1 read-to-write ratio)
```

Capacity Estimation

```
Assumptions:
- 100M new URLs per month
- 100:1 read/write ratio → 10B redirections/month

Write QPS: 100M / (30 x 24 x 3600) ≈ ~40 URLs/sec
Read QPS: 40 × 100 = ~4000 redirections/sec

Storage (5 years):
- 100M × 12 × 5 = 6 Billion records
- Each record ~500 bytes
- Total: 6B × 500B = ~3 TB

Bandwidth:
- Incoming: 40 × 500 bytes = 20 KB/s
- Outgoing: 4000 × 500 bytes = 2 MB/s

Cache Memory (80-20 rule):
- 20% of URLs generate 80% of traffic
- 4000 × 86400 × 0.2 × 500 bytes ≈ ~35 GB/day
```

High-Level Architecture

```
+-----------+   +----------------+   +--------------+   +---------------+
|  Client   |-->| Load Balancer  |-->| API Servers  |-->|  Cache        |
| (Browser) |<--| (Nginx/ALB)    |<--| (Stateless)  |<--| (Redis)       |
+-----------+   +----------------+   +------+-------+   +---------------+
                                            |                   |
                                            v                   |
                                  +-----------------+           |
                                  |   Database      |<----------+
                                  | (NoSQL/SQL)     |
                                  +--------+--------+
                                           |
                                  +--------v--------+
                                  | Analytics Svc   |
                                  +-----------------+
```

Detailed Component Design

◆ A. Short URL Generation Strategies

```
Strategy 1: Base62 Encoding of Auto-Increment ID
═══════════════════════════════════════════════

Characters: [a-z, A-Z, 0-9] = 62 characters
- 6 chars → 62^6 = ~56.8 Billion unique URLs
- 7 chars → 62^7 = ~3.5 Trillion unique URLs

Example:
  ID = 125   → Base62 = "cb"
  ID = 19158 → Base62 = "e9a"

  def encode_base62(num):
      chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
      base = len(chars)   # 62
      result = []
      while num > 0:
          result.append(chars[num % base])
          num //= base
      return ''.join(reversed(result))

  def decode_base62(short_str):
      chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"
      base = len(chars)
      num = 0
      for char in short_str:
          num = num * base + chars.index(char)
      return num

Pros: No collision, simple
Cons: Predictable (sequential), single point of failure for ID gen
```

```
Strategy 2: MD5/SHA256 Hash + First N Characters
═══════════════════════════════════════════════

Process: Long URL → MD5 Hash → Take first 7 chars (Base62 encoded)

Problem: Collisions possible!
Solution: If collision → append user_id or timestamp, rehash

Pros: Distributed (no central counter)
Cons: Collision handling needed
```

```
Strategy 3: Pre-Generated Key Service (KGS) □ RECOMMENDED
═══════════════════════════════════════════════════════

+-----------------------------------------------+
|           Key Generation Service              |
|                                               |
|  +--------------+    +--------------+         |
|  | Unused Keys  |--->|  Used Keys   |         |
|  |  Database    |    |  Database    |         |
|  |              |    |              |         |
|  |  abc123      |    |  xyz789      |         |
|  |  def456      |    |  mno321      |         |
|  +--------------+    +--------------+         |
+-----------------------------------------------+
         |
         v  (Keys loaded into memory in batches)
+---------------------------------------------+
|  App Server 1        App Server 2           |
|  [key1, key2]        [key3, key4]           |
|                                             |
|  Each server gets a batch of keys from KGS  |
|  → eliminates contention                    |
+---------------------------------------------+

6-char keys, 62 chars → 56.8B keys
Storage: 56.8B × 6 bytes = ~341 GB (manageable)

Pros: No collision, no contention, fast
Cons: Slightly more complex architecture
```

◆ B. Database Schema

```
-- SQL Schema
CREATE TABLE urls (
    id          BIGINT PRIMARY KEY AUTO_INCREMENT,
    short_code  VARCHAR(7) UNIQUE NOT NULL,
    long_url    VARCHAR(2048) NOT NULL,
    user_id     BIGINT,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at  TIMESTAMP,
    click_count BIGINT DEFAULT 0,
    INDEX idx_short_code (short_code),
    INDEX idx_user_id    (user_id)
);

-- NoSQL (DynamoDB) Schema
{
    "short_code": "abc123",        // Partition Key
    "long_url":   "https://...",
    "user_id":    "user_456",
    "created_at": 1699000000,
    "expires_at": 1730000000,
    "click_count": 15234
}
```

◆ C. API Design

```
POST /api/v1/shorten
────────────────────────────────────────────────────────
Request:
{
    "long_url":     "https://www.example.com/very/long/path?param=value",
    "custom_alias": "my-link",        // optional
    "expires_at":   "2025-12-31"      // optional
}

Response (201 Created):
{
    "short_url":  "https://short.ly/abc123",
    "long_url":   "https://www.example.com/very/long/path?param=value",
    "expires_at": "2025-12-31T00:00:00Z"
}

GET /{short_code}
────────────────────────────────────────────────────────
Response: HTTP 301 (Permanent) or 302 (Temporary) Redirect
Location: https://www.example.com/very/long/path?param=value

301 vs 302:
- 301: Browser caches redirect → less server load, no click tracking
- 302: Browser always hits server → more load, accurate analytics 

GET /api/v1/analytics/{short_code}
────────────────────────────────────────────────────────
Response:
{
    "short_code":       "abc123",
    "total_clicks":     15234,
    "clicks_by_country": {"US": 5000, "IN": 3000},
    "clicks_by_device":  {"mobile": 8000, "desktop": 7234}
}
```

◆ D. Read Flow (Redirection)

```
User visits https://short.ly/abc123

Step 1: Request hits Load Balancer
Step 2: Routes to API Server
Step 3: Check Redis Cache
        │
        ├── Cache HIT  → Return long URL immediately
        │
        └── Cache MISS
                │
Step 4: Query Database (by short_code)
Step 5: If found:
        ├── Store in Redis Cache (TTL: 24 hours)
        ├── Async: Increment click counter (Kafka → Analytics)
        └── Return 302 Redirect with long URL

        If not found:  Return 404 Not Found
        If expired:    Return 410 Gone
```

◆ E. Scaling Strategies

```
1. DATABASE SHARDING
═══════════════════
   Shard by first character of short_code:
   Shard 0: a-f  →  DB Server 1
   Shard 1: g-m  →  DB Server 2
   Shard 2: n-s  →  DB Server 3
   Shard 3: t-z  →  DB Server 4
   OR: Consistent Hashing on short_code

2. CACHING STRATEGY
═══════════════════
   - Cache hot URLs in Redis (80-20 rule)
   - Cache eviction: LRU (Least Recently Used)
   - Cache size: ~35 GB (fits in memory)
   Write-through for new URLs: Write to DB → Write to Cache
   Read path:     Cache → DB (on miss) → Populate Cache

3. DATABASE REPLICATION
═══════════════════════
   Master-Slave setup:
   - Master: handles writes
   - Slaves: handle reads (read replicas)
   - Since read-heavy (100:1), multiple read replicas help

4. ANALYTICS PIPELINE
═══════════════════
   Click events → Kafka → Stream Processing → Analytics DB
   (Don't update main DB on every click - use async processing)
```

2. Rate Limiter

Problem Statement

Control the rate of requests a client can send to an API within a given time window to protect services from abuse and ensure fair resource usage.

Functional Requirements

```
- Limit requests per client/IP/API key
- Configurable rate limits (e.g., 100 req/min, 1000 req/hour)
- Return appropriate response when limit exceeded (HTTP 429)
- Support multiple rate limiting rules simultaneously
- Rate limiting at different levels: per user, per IP, per API endpoint
```

Non-Functional Requirements

```
- Very low latency (added to every request path)
- Highly available (if rate limiter fails, should it allow or deny?)
- Distributed (works across multiple servers)
- Accurate counting (minimal race conditions)
- Memory efficient
```

Rate Limiting Algorithms

◆ Algorithm 1: Token Bucket □ (Most Popular)

```
Concept: A bucket holds tokens. Each request consumes a token.
         Tokens are added at a fixed rate. If bucket is empty → reject.

Parameters:
  - Bucket Size (max tokens): Controls burst capacity
  - Refill Rate: tokens added per second

  Bucket Capacity: 10 tokens
  Refill Rate:      2 tokens/second

  Time 0: 10 tokens available
  5 requests → 5 tokens left
  3 more     → 2 tokens left
  Wait 1s    → 4 tokens (2 refilled)
  Burst of 5 → 4 accepted, 1 rejected

class TokenBucket:
    def __init__(self, capacity, refill_rate):
        self.capacity = capacity
        self.tokens = capacity
        self.refill_rate = refill_rate  # tokens per second
        self.last_refill_time = time.time()

    def allow_request(self):
        self._refill()
        if self.tokens >= 1:
            self.tokens -= 1
            return True
        return False

    def _refill(self):
        now = time.time()
        elapsed = now - self.last_refill_time
        tokens_to_add = elapsed * self.refill_rate
        self.tokens = min(self.capacity, self.tokens + tokens_to_add)
        self.last_refill_time = now

Pros: □ Simple  □ Memory efficient  □ Allows bursts
Cons: □ Two parameters to tune
```

◆ Algorithm 2: Leaky Bucket

```
Concept: Requests enter a queue (bucket). Processed at a fixed rate.
         If queue is full, new requests are dropped.

class LeakyBucket:
    def __init__(self, capacity, leak_rate):
        self.capacity = capacity
        self.water = 0
        self.leak_rate = leak_rate
        self.last_check = time.time()

    def allow_request(self):
        self._leak()
        if self.water < self.capacity:
            self.water += 1
            return True
        return False

    def _leak(self):
        now = time.time()
        elapsed = now - self.last_check
        self.water = max(0, self.water - elapsed * self.leak_rate)
        self.last_check = now

Pros: □ Smooth output rate  □ Memory efficient
Cons: □ No burst handling   □ Old requests processed while new ones dropped
```

◆ Algorithm 3: Fixed Window Counter

```
Concept: Divide time into fixed windows. Count requests per window.

Problem — Boundary burst issue:
  Window 1 (0:00-0:59)          Window 2 (1:00-1:59)
  ____________________          ____________________

  Quiet.....|5 reqs at 0:59    |5 reqs at 1:00|.....Quiet
            └____________________┘

            10 requests in 2 seconds! (2× the limit)

Pros: □ Very simple  □ Memory efficient
Cons: □ Boundary burst problem (2× limit at window edges)
```

◆ Algorithm 4: Sliding Window Counter □ (Best Balance)

```
Concept: Combine Fixed Window + Sliding Window
         Weight counts from current and previous windows

Limit: 100 req/hour
Current time: 1:15 (25% into current window)
Previous window (0:00-0:59): 84 requests
Current window  (1:00-1:59): 36 requests

Weighted count = 84 × (1 - 0.25) + 36
               = 63 + 36 = 99

99 < 100 → ALLOWED
Next request → 100 → REJECTED

class SlidingWindowCounter:
    def __init__(self, limit, window_seconds):
        self.limit = limit
        self.window_seconds = window_seconds
        self.prev_count = 0
        self.curr_count = 0
        self.curr_window_start = 0

    def allow_request(self):
        now = time.time()
        curr_window = int(now / self.window_seconds)
        if curr_window != self.curr_window_start:
            self.prev_count = self.curr_count
            self.curr_count = 0
            self.curr_window_start = curr_window
        elapsed_ratio = (now % self.window_seconds) / self.window_seconds
        weighted = self.prev_count * (1 - elapsed_ratio) + self.curr_count
        if weighted >= self.limit:
            return False
        self.curr_count += 1
        return True

Pros: □ Smooth  □ Memory efficient  □ Good accuracy
Cons: □ Approximate (not 100% exact)
```

Distributed Rate Limiting Architecture

```
+-----------------------------------------------+
|         DISTRIBUTED RATE LIMITER              |
|                                               |
|  Client1  Client2  Client3                    |
|     |        |        |                       |
|  +--+--------+--------+---+                   |
|  |     Load Balancer       |                  |
|  +--+--------+--------+---+                   |
|     |        |        |                       |
|  Server1  Server2  Server3                    |
|  (Rate    (Rate    (Rate                      |
|   Limiter) Limiter) Limiter)                  |
|     |        |        |                       |
|     +--------+--------+                       |
|              |                                |
|     +---------+---------+                    |
|     |   Redis Cluster    |  <- Centralized    |
|     |                    |     Counter        |
|     |  user_123: 45      |                    |
|     |  user_456: 89      |                    |
|     +--------------------+                    |
|                                               |
|  Rules Engine:                                |
|   Rule 1: /api/*  → 100 req/min              |
|   Rule 2: /login  →   5 req/min              |
|   Rule 3: Premium → 1000 req/min             |
+-----------------------------------------------+
```

Redis Lua Script (Atomic Rate Limiting)

```lua
-- Atomic sliding window rate limiter
-- Ensures no race conditions

local key    = KEYS[1]              -- "rate_limit:user_123"
local limit  = tonumber(ARGV[1])    -- 100
local window = tonumber(ARGV[2])    -- 60 (seconds)
local now    = tonumber(ARGV[3])    -- current timestamp

-- Remove old entries outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count current requests in window
local count = redis.call('ZCARD', key)

if count < limit then
    redis.call('ZADD', key, now, now .. ':' .. math.random())
    redis.call('EXPIRE', key, window)
    return 1   -- ALLOWED
else
    return 0   -- REJECTED
end
```

HTTP Response Headers

```
When ALLOWED (HTTP 200):
  X-RateLimit-Limit:     100   -- Max requests allowed
  X-RateLimit-Remaining:  45   -- Remaining in window
  X-RateLimit-Reset: 1699000060 -- Unix timestamp when limit resets

When REJECTED (HTTP 429 Too Many Requests):
  X-RateLimit-Limit:       100
  X-RateLimit-Remaining:     0
  Retry-After:              30  -- Seconds to wait
  Body: { "error": "Rate limit exceeded", "retry_after": 30 }
```

3. Distributed Cache (like Redis)

Problem Statement

Design a distributed caching system that stores key-value pairs in memory across multiple nodes, providing sub-millisecond read/write latency.

Functional Requirements

```
- PUT(key, value) - Store a key-value pair
- GET(key) - Retrieve value by key
- DELETE(key) - Remove a key-value pair
- TTL support (key expiration)
- Support for multiple data types (strings, lists, sets, hashes)
- Atomic operations (INCR, DECR, CAS)
```

Non-Functional Requirements

```
- Sub-millisecond latency (<1ms for reads)
- High throughput (100K+ ops/second per node)
- High availability (minimal downtime)
- Scalable (add nodes to increase capacity)
- Fault tolerant (handle node failures gracefully)
```

Key Design Components

◆ A. Data Partitioning — Consistent Hashing

```
Problem with Simple Hash: node = hash(key) % N
  If N changes (node added/removed), almost ALL keys get remapped!

Consistent Hashing Solution:
═══════════════════════════════

Imagine a circle (ring) with positions 0 to 2^32.

Nodes are placed on the ring: hash(node_id)
Keys are placed using:        hash(key)
A key belongs to the FIRST node found clockwise.

Example:
  hash("Node-A") = 50   → position 50
  hash("Node-B") = 150  → position 150
  hash("Node-C") = 250  → position 250

  hash("user:123") = 80  → Node-B (first node after 80)
  hash("user:456") = 200 → Node-C
  hash("user:789") = 300 → Node-A (wraps around)

Virtual Nodes (Vnodes):
  Each physical node gets multiple virtual positions (100-200 typical)
  Node-A → [V_A1=30, V_A2=120, V_A3=270]
  → Much more uniform distribution

When Node-B is removed:
  Only keys that were on Node-B are redistributed.
  Other keys are NOT affected! (Unlike simple hash)

class ConsistentHashRing:
    def __init__(self, nodes, virtual_nodes=150):
        self.ring = SortedDict()
        self.virtual_nodes = virtual_nodes
        for node in nodes:
            self.add_node(node)

    def add_node(self, node):
        for i in range(self.virtual_nodes):
            pos = hash(f"{node}:vn{i}") % (2**32)
            self.ring[pos] = node

    def get_node(self, key):
        if not self.ring: return None
        position = hash(key) % (2**32)
        idx = self.ring.bisect_right(position)
        if idx == len(self.ring): idx = 0
        return self.ring.values()[idx]
```

◆ B. Cache Eviction Policies

```
1. LRU (Least Recently Used) □ Most Common
═══════════════════════════════════════════

Uses: Doubly Linked List + HashMap for O(1) operations

   HashMap:                    O(1) lookup
     key1 → node1
     key2 → node2

   Doubly Linked List:         O(1) insert/delete
   HEAD <-> [key3] <-> [key1] <-> [key2] <-> TAIL
   (Most Recent)              (Least Recent)

   GET(key1): Move key1 to head
   Cache Full + PUT(key4): Remove TAIL (LRU), add key4 to head

class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache: return None
        self.cache.move_to_end(key)   # Mark as recently used
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)  # Remove least recent

2. LFU (Least Frequently Used)
   Evict keys with lowest access frequency
   Tie-breaker: LRU among same frequency

3. TTL-based Expiration (Redis approach)
   - On access: Check if expired → delete if yes
   - Every 100ms: Randomly sample 20 keys, delete expired
   - If >25% were expired: repeat immediately
```

◆ C. Cache Patterns

```
Pattern 1: Cache-Aside (Lazy Loading) □ Most Common
═══════════════════════════════════════════════════

Read:
  1. Check cache
  2. If HIT  → return cached data
  3. If MISS → read from DB → write to cache → return

Write:
  1. Write to DB
  2. Invalidate cache (delete key)
  (NOT write to cache - avoids stale data if write fails)

def get_user(user_id):
    cached = cache.get(f"user:{user_id}")
    if cached: return cached                # Cache HIT
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

def update_user(user_id, data):
    db.update("UPDATE users SET ... WHERE id = ?", user_id)
    cache.delete(f"user:{user_id}")         # Invalidate

Pattern 2: Write-Through
═══════════════════════
Every write goes to cache AND DB synchronously.
Pros: Cache always consistent with DB
Cons: Higher write latency (2 writes)

Pattern 3: Write-Behind (Write-Back)
═══════════════════════════════════
Write to cache immediately, async write to DB later.
Pros: Very low write latency
Cons: Data loss risk if cache node fails before DB write
```

◆ D. Cache Stampede (Thundering Herd)

```
Scenario: Popular key expires → 1000 requests hit DB simultaneously

Solutions:

1. Locking (Mutex):
   Only ONE request fetches from DB, others wait

   def get_with_lock(key):
       value = cache.get(key)
       if value: return value

       lock_key = f"lock:{key}"
       if cache.setnx(lock_key, 1, ttl=5):  # Acquire lock
           try:
               value = db.query(key)
               cache.set(key, value, ttl=3600)
           finally:
               cache.delete(lock_key)       # Release lock
           return value
       else:
           time.sleep(0.1)
           return get_with_lock(key)        # Retry

2. Early Expiration:
   Refresh cache before it actually expires
   Background job refreshes at 3000s (before 3600s expiry)

3. Probabilistic Early Recomputation:
   Each request has a small chance of refreshing near expiry
```

◆ E. Replication & Fault Tolerance

```
1. Master-Slave (Primary-Replica)
   Writes → Master only
   Reads  → Master or any Replica
   Failure → Promote Replica to Master

   Master ──async──> Replica 1
   Master ──async──> Replica 2

2. Multi-Master (Active-Active)
   Both accept writes → Conflict resolution needed
   Last-Writer-Wins or Vector Clocks

3. Leaderless (Quorum-based)
   Write to W nodes, Read from R nodes
   W + R > N ensures overlap (consistency)
   N=3, W=2, R=2 → At least 1 node has latest data
```

4. Message Queue System (like Apache Kafka)

Problem Statement

Design a distributed messaging system that allows producers to publish messages and consumers to subscribe and process messages reliably with ordering guarantees.

Functional Requirements

```
- Producers can publish messages to named topics
- Consumers can subscribe to topics and read messages
- Messages are persisted (not lost if consumers are slow)
- Ordering guarantees (at least within a partition)
- Consumer groups (distribute work among consumers)
- Message replay (consumers can re-read old messages)
- At-least-once / at-most-once / exactly-once semantics
```

Non-Functional Requirements

```
- High throughput (millions of messages/second)
- Low latency (<10ms for publishing)
- Durability (messages survive node failures)
- Scalability (add more brokers as load grows)
- Fault tolerance (no single point of failure)
```

Core Concepts

◆ A. Topics and Partitions

```
TOPIC: A named category/feed of messages
PARTITION: A topic is split into partitions for parallelism

Topic "orders" with 3 partitions:
═══════════════════════════════════

Partition 0: [msg0] [msg3] [msg6] [msg9]  ← ordered within partition
              offset: 0    1      2     3

Partition 1: [msg1] [msg4] [msg7]
              offset: 0    1      2

Partition 2: [msg2] [msg5] [msg8]
              offset: 0    1      2

Key points:
- Messages within a partition are STRICTLY ordered
- No ordering guarantee ACROSS partitions
- Each partition is an append-only, immutable commit log
- Offset = unique sequential ID within a partition

Partition Assignment:
- With key:    partition = hash(key) % num_partitions
  (Same key always → same partition → ordering guaranteed for that key)
- Without key: Round-robin assignment

Example:
  Producer sends order for user_123:
    hash("user_123") % 3 = 1 → goes to Partition 1
  All orders for user_123 always go to Partition 1 → ORDER PRESERVED
```

◆ B. Storage Design

```
Each partition is stored as a COMMIT LOG on disk:

Partition 0 Directory:
  00000000000000000000.log    (segment, offsets 0-999)
  00000000000000000000.index  (offset → file position)
  00000000000000001000.log    (segment, offsets 1000-1999)
  00000000000000002000.log    (active segment)

Message Format:
  Offset     (8 bytes)
  Timestamp  (8 bytes)
  Key Length (4 bytes)
  Key        (variable)
  Value Len  (4 bytes)
  Value      (variable)
  CRC        (4 bytes)

Why Sequential Disk I/O is Fast:
- Sequential reads: ~600 MB/s (HDD), 2+ GB/s (SSD)
- Kafka exploits this by always APPENDING
- OS page cache handles caching automatically
```

◆ C. Replication for Fault Tolerance

```
Topic "orders", Partition 0, Replication Factor = 3:

  Broker 1           Broker 2           Broker 3
  P0: LEADER         P0: FOLLOWER       P0: FOLLOWER
  [m1, m2, m3]       [m1, m2, m3]       [m1, m2] ← lagging

ISR (In-Sync Replicas):
  Replicas that are caught up with the leader
  P0 ISR = {Broker1(leader), Broker2}   (Broker3 lagging)

acks Configuration:
  acks=0:   Don't wait for any ack   (fastest,  least safe)
  acks=1:   Wait for leader only     (balanced)
  acks=all: Wait for all ISR replicas (slowest,  safest) □

Leader Failure:
  1. Broker 1 goes down
  2. ZooKeeper detects failure
  3. Controller selects new leader from ISR (Broker 2)
  4. Producers/Consumers redirect to Broker 2
```

◆ D. Consumer Groups

```
Consumer Group: A set of consumers cooperating to consume a topic.

Topic "orders" with 4 partitions:

Scenario: 2 consumers (each gets 2 partitions)
  P0, P1 → Consumer 1
  P2, P3 → Consumer 2

Scenario: 4 consumers (ideal 1:1 mapping)
  P0 → C1  |  P1 → C2  |  P2 → C3  |  P3 → C4

Scenario: 5 consumers (one is IDLE!)
  Rule: Max useful consumers = Number of partitions

Multiple Consumer Groups:
  Each group gets its OWN independent copy of all messages.
  Group A (Order Processing): reads m1, m2, m3 from P0
  Group B (Analytics):        reads m1, m2, m3 from P0 independently
```

◆ E. Delivery Guarantees

```
1. At-Most-Once
   Commit offset BEFORE processing → message can be lost on crash
   Use case: Metrics where occasional loss is acceptable

2. At-Least-Once □ (Most Common)
   Process BEFORE committing offset → possible re-delivery on crash
   Application must be IDEMPOTENT

   while True:
       messages = consumer.poll(timeout=1000)
       for msg in messages:
           process(msg)        # Process first
       consumer.commit()       # Then commit offset

3. Exactly-Once (Hardest)
   Idempotent producers + Transactional produce+consume

   BEGIN TRANSACTION
     result = process(message_42)
     save_result(result)
     save_offset(42)
   COMMIT TRANSACTION

   Use case: Financial transactions, exact counting
```

5. Distributed Job Scheduler

Problem Statement

Design a system to schedule and execute jobs at specified times or intervals, distributed across multiple worker nodes.

Functional Requirements

```
- Schedule one-time jobs for future execution
- Schedule recurring jobs (cron-like: "every 5 min", "daily at 3 AM")
- Cancel / pause / resume scheduled jobs
- Job prioritization
- Job dependencies (run job B after job A completes)
- Retry failed jobs with configurable policies
- Job status tracking and logging
```

Non-Functional Requirements

```
- Exactly-once execution (a job should NOT run twice)
- Scalable (millions of scheduled jobs)
- Fault tolerant (jobs execute even if some nodes fail)
- Low scheduling latency (jobs fire within seconds)
- Persistent (jobs survive system restarts)
```

Data Models

```
CREATE TABLE jobs (
    job_id          UUID PRIMARY KEY,
    job_name        VARCHAR(255) NOT NULL,
    job_type        ENUM('ONE_TIME', 'RECURRING'),
    status          ENUM('SCHEDULED', 'RUNNING', 'COMPLETED',
                         'FAILED', 'CANCELLED', 'PAUSED'),
    priority        INT DEFAULT 5,       -- 1 (highest) to 10 (lowest)

    -- Scheduling
    scheduled_time  TIMESTAMP,           -- For one-time jobs
    cron_expression VARCHAR(100),        -- "0 */5 * * *"
    next_run_time   TIMESTAMP,           -- Next execution time
    timezone        VARCHAR(50),

    -- Execution
    handler_class   VARCHAR(255),        -- "com.app.EmailJob"
    payload         JSON,

    -- Retry config
    max_retries     INT DEFAULT 3,
    retry_count     INT DEFAULT 0,
    retry_delay_ms  INT DEFAULT 1000,    -- Exponential backoff base

    INDEX idx_next_run (status, next_run_time),  -- Critical!
    INDEX idx_status (status)
);
```

Scheduling Mechanism

```
class Scheduler:
    def run(self):
        while True:
            # 1. Find jobs due for execution
            due_jobs = db.query("""
                SELECT * FROM jobs
                WHERE status = 'SCHEDULED'
                  AND next_run_time <= NOW()
                ORDER BY priority ASC, next_run_time ASC
                LIMIT 100
                FOR UPDATE SKIP LOCKED    -- Prevents double pickup!
            """)

            for job in due_jobs:
                # 2. Mark as RUNNING
                db.update("UPDATE jobs SET status='RUNNING' WHERE job_id=?",
                          job.id)

                # 3. Push to message queue for workers
                queue.publish(
                    topic="job-executions",
                    message={"job_id": job.id, "handler": job.handler_class,
                             "payload": job.payload}
                )

                # 4. For recurring jobs, schedule next run
                if job.job_type == 'RECURRING':
                    next_time = calculate_next_cron(job.cron_expression)
                    db.update("UPDATE jobs SET next_run_time=?, status='SCHEDULED'"
                              " WHERE job_id=?", next_time, job.id)

            time.sleep(1)   # Poll every second
```

Exactly-Once Execution

```
Problem: Worker crashes mid-execution → job gets requeued → runs again!

Solutions:

1. Database-Level Locking:
   SELECT ... FOR UPDATE SKIP LOCKED
   → Only one scheduler/worker can claim a job

2. Distributed Lock (Redis):
   lock = redis.set(f"job_lock:{job_id}", worker_id, NX, EX=300)
   if not lock: return  # Another worker has it
   try:
       execute_job(job)
   finally:
       redis.delete(f"job_lock:{job_id}")

3. Idempotency Token:
   def execute(job, execution_id):
       if db.exists("executed_jobs", execution_id):
           return   # Already done
       result = do_work(job)
       db.insert("executed_jobs", execution_id, result)  # Atomic
```

Retry Strategy

```
Exponential Backoff with Jitter:
═══════════════════════════════════

Attempt 1: Failed → retry after  1s
Attempt 2: Failed → retry after  2s
Attempt 3: Failed → retry after  4s
Attempt 4: Failed → retry after  8s + random jitter
Attempt 5: Failed → Dead Letter Queue (DLQ)

def calculate_retry_delay(retry_count, base_ms=1000):
    delay   = base_ms * (2 ** retry_count)
    delay   = min(delay, 60000)          # Cap at 1 minute
    jitter  = random.uniform(0, delay * 0.1)
    return delay + jitter

Dead Letter Queue (DLQ):
  Jobs that fail all retries → DLQ for manual investigation
```

6. Notification System

Problem Statement

Design a system to send notifications to users across multiple channels (push, SMS, email) with high reliability at scale.

Functional Requirements

```
- Send via: Push (iOS/Android), SMS, Email, In-App
- Support types: transactional, marketing, alerts
- User preferences (opt-in/out per channel)
- Template management
- Scheduling (send later, timezone-aware)
- Rate limiting (don't spam users)
- Delivery tracking and analytics
- Priority levels (critical vs marketing)
```

Detailed Notification Flow

```
Example: User places order → Send order confirmation

Step 1: Order Service calls Notification API
  POST /api/v1/notifications
  {
    "user_id": "user_123",
    "notification_type": "ORDER_CONFIRMATION",
    "channels": ["push", "email"],
    "priority": "HIGH",
    "template_id": "order_confirm_v2",
    "template_data": {
      "order_id":      "ORD-456",
      "items":         ["iPhone 15", "AirPods"],
      "total":         "$1,299.00",
      "delivery_date": "Dec 25, 2024"
    },
    "idempotency_key": "order-456-confirm"   // Prevent duplicates
  }

Step 2: Validation & Enrichment
  ├── Validate request (required fields, valid user)
  ├── Check idempotency (sent before?)
  ├── Rate limit check (user over-notified?)
  ├── Fetch user preferences:
  │     email: { enabled: true }
  │     sms:   { enabled: false }  ← User opted out
  │     push:  { enabled: true, tokens: ["fcm_token_abc"] }
  │     quiet_hours: { start: "22:00", end: "07:00", tz: "US/Pacific" }
  ├── Filter channels: Remove SMS
  └── Render templates for each channel

Step 3: Template Rendering
  Email Subject: "Your order ORD-456 is confirmed! □"
  Push Body:     "Your order ORD-456 for $1,299.00 is on its way!"

Step 4: Push to Priority Queue → Channel Workers → FCM/APNS/SES
```

Data Model

```
CREATE TABLE notifications (
    notification_id UUID PRIMARY KEY,
    user_id         VARCHAR(50),
    type            VARCHAR(50),     -- ORDER_CONFIRM, PROMO, ALERT
    priority        ENUM('CRITICAL', 'HIGH', 'MEDIUM', 'LOW'),
    status          ENUM('PENDING', 'SENT', 'DELIVERED', 'READ', 'FAILED'),
    created_at      TIMESTAMP,
    idempotency_key VARCHAR(100) UNIQUE
);

CREATE TABLE notification_deliveries (
    delivery_id     UUID PRIMARY KEY,
    notification_id UUID,
    channel         ENUM('PUSH', 'SMS', 'EMAIL', 'IN_APP'),
    status          ENUM('QUEUED','SENT','DELIVERED','OPENED','FAILED'),
    provider        VARCHAR(50),    -- FCM, APNS, Twilio, SES
    sent_at         TIMESTAMP,
    delivered_at    TIMESTAMP,
    retry_count     INT DEFAULT 0
);

CREATE TABLE device_tokens (
    token_id     UUID PRIMARY KEY,
    user_id      VARCHAR(50),
    platform     ENUM('IOS', 'ANDROID', 'WEB'),
    token        VARCHAR(500),
    is_active    BOOLEAN DEFAULT TRUE,
    last_used_at TIMESTAMP
);
```

Handling Scale (Bulk Notifications)

```
Bulk Marketing to 10M users — Fan-out Pattern:
═══════════════════════════════════════════════

Step 1: Create bulk job
  { "type": "BULK", "segment": "active_users_last_30_days",
    "template_id": "holiday_sale", "estimated_count": 10_000_000 }

Step 2: Segment workers fetch users in batches of 10,000

Step 3: Each batch → 20,000 messages (push + email)

Step 4: Rate limited: max 1000 emails/sec
  → 10M / 1000 = ~2.7 hours to complete

Step 5: Progress tracking
  { "total": 10_000_000, "sent": 7_500_000,
    "delivered": 7_200_000, "failed": 50_000, "progress": "75%" }
```

7. Search Engine

Problem Statement

Design a full-text search engine that can index documents and return relevant results for search queries with low latency.

Functional Requirements

```
- Index documents (web pages, products, articles)
- Full-text search queries
- Relevance ranking of results
- Filters (category, date range, price range)
- Auto-complete / search suggestions
- Spell correction ("did you mean?")
- Pagination of results
- Boolean queries (AND, OR, NOT)
```

Non-Functional Requirements

```
- Search latency < 200ms
- High availability
- Scalable to billions of documents
- Near real-time indexing (new docs searchable within seconds)
- Thousands of QPS
```

Core Data Structure: Inverted Index

```
Forward Index (what a document contains):
  Doc1: "Redis is a fast in-memory cache"
  Doc2: "Memcached is also a fast cache"
  Doc3: "Redis supports data structures"

Inverted Index (which documents contain a word):
═══════════════════════════════════════════════

  Term        │  Posting List
  ────────────┼──────────────────────────────────────
  redis       │  [(Doc1, pos=0, tf=1), (Doc3, pos=0, tf=1)]
  fast        │  [(Doc1, pos=3, tf=1), (Doc2, pos=3, tf=1)]
  cache       │  [(Doc1, pos=6, tf=1), (Doc2, pos=5, tf=1)]
  memory      │  [(Doc1, pos=5, tf=1)]
  memcached   │  [(Doc2, pos=0, tf=1)]
  data        │  [(Doc3, pos=2, tf=1)]
  structures  │  [(Doc3, pos=3, tf=1)]

Search "redis cache":
  "redis" → [Doc1, Doc3]
  "cache" → [Doc1, Doc2]
  AND intersection: [Doc1]
  OR union:         [Doc1, Doc2, Doc3]
```

Indexing Pipeline

```
Document → Tokenization → Normalization → Indexing

Step 1: Tokenization
  Input:  "Redis is a Fast In-Memory Cache System!"
  Output: ["Redis", "is", "a", "Fast", "In-Memory", "Cache", "System"]

Step 2: Normalization
  a) Lowercasing:      ["redis", "is", "a", "fast", "cache", "system"]
  b) Stop word removal: ["redis", "fast", "cache", "system"]
                        (Remove: is, a, the, in, of, and...)
  c) Stemming:          "caches"→"cache", "systems"→"system"
  d) Synonyms:          "fast" → also index as "quick", "rapid"

Step 3: Build Inverted Index
  For each token: add (doc_id, position, tf) to posting list

class InvertedIndex:
    def add_document(self, doc_id, text):
        tokens = self.normalize(self.tokenize(text))
        term_freq = Counter(tokens)
        for pos, term in enumerate(tokens):
            if term not in self.index: self.index[term] = []
            self.index[term].append({'doc_id': doc_id,
                                     'position': pos,
                                     'tf': term_freq[term]})

    def search(self, query, op='AND'):
        terms = self.normalize(self.tokenize(query))
        sets = [set(p['doc_id'] for p in self.index.get(t,[]))
                for t in terms]
        return set.intersection(*sets) if op=='AND' else set.union(*sets)
```

Relevance Ranking: TF-IDF

```
TF-IDF = Term Frequency × Inverse Document Frequency
═══════════════════════════════════════════════════

TF(t, d)  = count(t in d) / total_terms_in_d
IDF(t)    = log(N / df(t))    (N=total docs, df=docs containing t)

Why IDF matters:
  "redis"  in 10/1000 docs → IDF = log(100)  = 2.0  (rare, relevant)
  "the"    in 900/1000 docs → IDF = log(1.11) = 0.1  (common, ignored)

Example: Search "redis cache performance"
  Doc1: "Redis is the fastest cache"
  Doc2: "Redis cache performance benchmarks"   ← matches all 3 terms
  Doc3: "How to improve cache performance"

  Scores:     Doc1=0.55  Doc2=0.85□  Doc3=0.53
  Result: Doc2 ranks highest
```

Sharding Strategy

```
Option: Document-Based Sharding □
═══════════════════════════════════
  Shard 1: Docs 1-1M    (complete index for these docs)
  Shard 2: Docs 1M-2M
  Shard 3: Docs 2M-3M

  Search → All shards simultaneously (scatter)
  Each shard returns Top 10
  Merge & Re-rank → Return global Top 10

  Pros: Each shard independent, easy to add docs
  Cons: Every query hits all shards
```

Auto-Complete

```
Data Structure: Trie (Prefix Tree)

                   root
                 /      \
                r         c
               /           \
              e             a
             /               \
            d                 c
           / \                 \
          i   u                h
         /     \                \
        s       c                e
       /         \
  "redis"    "reduce"         "cache"

Search "red":
  Navigate: root → r → e → d
  Return all words below 'd':  ["redis" (100), "reduce" (30)]
  Sorted by score (popularity)

For scale: Pre-computed suggestions in Redis
  "r"   → ["redis", "react", "rest", "ruby"]
  "re"  → ["redis", "react", "rest"]
  "red" → ["redis", "reduce", "reddit"]
```

8. File Storage System (like Amazon S3)

Problem Statement

Design a distributed object storage system that can store and retrieve files of any size with high durability, availability, and scalability.

Functional Requirements

```
- Upload / Download / Delete files
- List files in a bucket/directory
- Metadata support (content-type, custom metadata)
- Versioning (multiple versions of a file)
- Access control (public, private, signed URLs)
- Multipart upload (for large files)
- Support files from 1 byte to 5 TB
```

Non-Functional Requirements

```
- 99.999999999% (11 nines) durability
- 99.99% availability
- Support billions of objects
- Low latency for reads (<100ms for metadata)
- High throughput for data transfer
- Strong consistency (read-after-write)
```

Upload Flows

```
Small File Upload (<64MB):
═══════════════════════════

Client → PUT /bucket/key + data
  → API Server: Generate object_id
  → Select storage nodes (Consistent Hashing)
  → Write chunk to Node 1, Node 2, Node 3
  → Store metadata (object_id, chunk_locations, checksum)
  → Return 200 OK (ETag)


Large File Upload (Multipart, >64MB):
═══════════════════════════════════════

Step 1: Initiate
  POST /bucket/video.mp4?uploads
  Response: { "upload_id": "upload_xyz" }

Step 2: Upload parts in PARALLEL (e.g., 8 parts of 64MB)
  PUT /bucket/video.mp4?uploadId=upload_xyz&partNumber=1
  PUT /bucket/video.mp4?uploadId=upload_xyz&partNumber=2
  ...  (each part returns an ETag)

Step 3: Complete
  POST /bucket/video.mp4?uploadId=upload_xyz
  Body: [{"partNumber": 1, "etag": "..."}, ...]

Benefits:
  - Parallel upload → faster transfer
  - Resume on failure (only re-upload failed parts)
  - Upload files larger than a single request limit
```

Data Durability

```
1. Replication (Simple)
────────────────────────

   Store 3 copies on different nodes/racks/AZs

   AZ-1: Copy 1    AZ-2: Copy 2    AZ-3: Copy 3

   Prob all 3 fail = (0.01)^3 = 0.000001%
   Cost: 3× storage (300% overhead)

2. Erasure Coding □ (Space Efficient)
──────────────────────────────────────

   Reed-Solomon (4, 2):
   - 4 data chunks + 2 parity chunks
   - Can tolerate ANY 2 failures and still recover
   - Storage overhead: 6/4 = 1.5× (vs 3× for replication)

   [D1] [D2] [D3] [D4] [P1] [P2]
    ↓    ↓    ↓    ↓    ↓    ↓
   N1   N2   N3   N4   N5   N6

   If N2 and N5 fail:
   → Use D1, D3, D4, P2 to reconstruct D2

3. Integrity Verification
──────────────────────────

   Every chunk has a SHA-256 checksum.
   Background process scans every chunk every ~2 weeks.
   If corrupted → re-replicate from healthy copy.
```

Storage Classes (Cost Optimization)

```
  Storage Class   | Durability  | Latency | Cost/GB/month
  ────────────────┼─────────────┼─────────┼────────────────
  STANDARD        | 11 nines    | ms      | $0.023  (Frequent access)
  INFREQUENT (IA) | 11 nines    | ms      | $0.0125 (Higher retrieval)
  ARCHIVE         | 11 nines    | hours   | $0.004  (Cold storage)

Lifecycle Policies:
  "Move photos >90 days old to IA"
  "Move logs >1 year to ARCHIVE"
  "Delete tmp/ files after 7 days"
```

API Design

```
Bucket Operations:
  PUT    /buckets/{name}             Create bucket
  DELETE /buckets/{name}             Delete bucket
  GET    /buckets/{name}?list        List objects (prefix filter)

Object Operations:
  PUT    /buckets/{bucket}/{key}     Upload object
  GET    /buckets/{bucket}/{key}     Download object
         (Range: bytes=0-1023)       Partial/range download
  DELETE /buckets/{bucket}/{key}     Delete object
  HEAD   /buckets/{bucket}/{key}     Metadata only

Pre-signed URLs (Temporary Access):
  POST /buckets/{bucket}/{key}/presign
  { "operation": "GET", "expires_in": 3600 }

  Response:
  { "url": "https://storage.example.com/bucket/key?sig=abc&exp=..." }

  Anyone with this URL can download for 1 hour — no auth needed.
```

Summary — All 8 Systems Compared

This final section provides a quick-reference comparison of all systems covered in this guide.

System	Key Pattern	Key Technology	Key Challenge
URL Shortener	Read-heavy 100:1 R:W	Base62/KGS Redis Cache	Unique ID gen No collisions
Rate Limiter	Every request path	Token Bucket Sliding Window	Distributed counting
Distributed Cache	Key-Value lookups	Consistent Hashing LRU Policy	Cache stampede Invalidation
Message Queue	Pub-Sub Decoupling	Commit Log Consumer Groups	Ordering + Exactly-once
Job Scheduler	Time-based execution	Priority Queue Dist. Locks	Exactly-once execution
Notification System	Fan-out Multi-channel	Channel Workers Kafka Queues	Multi-channel Scale & prefs
Search Engine	Full-text search	Inverted Index TF-IDF Scoring	Relevance ranking
File Storage	Object store BLOB storage	Erasure Coding Chunking	Durability (11 nines)

Each of these systems demonstrates fundamental distributed systems principles: Partitioning, Replication, Consistency, Availability, and Fault Tolerance — the universal building blocks for any large-scale system design.